

Diseño e Implementación de un Analizador Sintáctico para un Lenguaje de Consultas de Productos e Inventario

^a Claudio M. F., Castillo; ^b Fernández, Facundo Nicolás; ^c Roch, Ignacio Fernando;
^d Rodríguez, Priscila Oriana; ^e Díaz, Edith Nancy Adela; ^f Cabrera, Wilson Alexis

^a UNNE, Teoría de la computación, ccmf.dev@gmail.com

^b UNNE, Teoría de la computación, facufer00@gmail.com

^c UNNE, Teoría de la computación, ignacioroch98@gmail.com

^d UNNE, Teoría de la computación, rodriguezpris10@gmail.com

^e UNNE, Teoría de la computación, adediaz18@gmail.com

^f UNNE, Teoría de la computación, wilsoncabrera.ok@gmail.com

Resumen

Se presenta el diseño de un lenguaje formal orientado a consultas de productos e inventario. El objetivo es permitir que una persona usuaria formule búsquedas mediante expresiones cercanas al lenguaje cotidiano, tales como “buscar producto mate imperial” o “mostrar productos con precio > 5000”, y que el sistema determine si dichas expresiones son válidas. Se definió una gramática libre de contexto en notación BNF, un analizador léxico para reconocer tokens y un analizador sintáctico ascendente LALR(1) implementado mediante PLY. La propuesta contempla la generación de una representación estructurada de la consulta, equivalente a un árbol sintáctico abstracto, y el reporte de errores orientado al usuario. Se corrigió un prefijo común en las producciones de filtros para evitar conflictos de decisión. Finalmente, se incluyen casos de prueba válidos e inválidos que permiten verificar el alcance del lenguaje. Las pruebas realizadas muestran que el sistema reconoce correctamente consultas válidas y detecta errores sintácticos en entradas inválidas.

1. Introducción

Los sistemas de comercio electrónico requieren consultas frecuentes sobre productos, categorías, precios y disponibilidad. Con el fin de simplificar este proceso, se diseñó un lenguaje específico de dominio que permite expresar consultas mediante frases cercanas al lenguaje natural.

El objetivo del trabajo es desarrollar un analizador léxico y sintáctico capaz de reconocer dichas consultas, validar su estructura y generar una representación interna de la información ingresada. Para ello se definió una gramática libre de contexto, se implementó un analizador léxico para el reconocimiento de tokens y un parser LALR(1) utilizando la biblioteca PLY.

La solución propuesta permite realizar búsquedas de productos, consultas de stock y filtrado por categorías o precios, proporcionando además mecanismos de detección de errores sintácticos.

2. Marco teórico

Una *gramática libre de contexto (GLC)* es un formalismo utilizado para describir la sintaxis de un lenguaje. Formalmente se representa como: $G = (N, T, P, S)$, donde N es el conjunto de símbolos no terminales, T el conjunto de símbolos terminales, P el conjunto de producciones y S el símbolo inicial de la gramática.

El proceso de análisis de una entrada se divide en dos etapas principales. En primer lugar, el *analizador léxico* transforma una secuencia de caracteres en una secuencia de tokens significativos para el lenguaje, tales como palabras reservadas, identificadores, números y operadores. Posteriormente, el *analizador sintáctico* verifica que la secuencia de tokens obtenida respete las reglas establecidas por la gramática.

Existen diferentes estrategias para implementar analizadores sintácticos. Entre ellas se encuentran los parsers predictivos LL(1), basados en análisis

descendente, y los parsers LR, basados en análisis ascendente. En este trabajo se seleccionó la estrategia LALR(1) mediante la biblioteca *PLY (Python Lex-Yacc)*, debido a que permite construir analizadores sintácticos robustos a partir de gramáticas libres de contexto y resolver automáticamente el análisis mediante autómatas y tablas de parsing.

Como resultado del análisis sintáctico puede construirse un *Árbol Sintáctico Abstracto (AST)*, una representación jerárquica de la estructura lógica de una consulta. El AST facilita el procesamiento posterior de la información al abstraer detalles sintácticos y conservar únicamente los elementos relevantes para la interpretación de la consulta.

3. Diseño de la gramática

El lenguaje diseñado permite realizar tres acciones principales: buscar un producto por nombre, consultar el stock de un producto y mostrar productos aplicando filtros. Asimismo, admite la combinación de múltiples filtros mediante los conectores lógicos "y" y "o", permitiendo expresar consultas más complejas.

3.1 Gramática Formal

$S \rightarrow \text{CONSULTA}$

$\text{CONSULTA} \rightarrow \text{BUSQUEDA} \mid \text{STOCK}$

$\text{BUSQUEDA} \rightarrow \text{buscar producto NOMBRE} \mid \text{mostrar productos CONDICION}$

$\text{STOCK} \rightarrow \text{ver stock NOMBRE}$

$\text{CONDICION} \rightarrow \text{FILTRO RESTO_CONDICION}$

$\text{RESTO_CONDICION} \rightarrow y \text{ FILTRO}$

$\text{RESTO_CONDICION} \mid o \text{ FILTRO RESTO_CONDICION} \mid \lambda$

$\text{FILTRO} \rightarrow \text{sin stock} \mid \text{con RESTO_CON} \mid \text{de categoria NOMBRE}$

$\text{RESTO_CON} \rightarrow \text{stock} \mid \text{precio COMPARADOR NUMERO}$

$\text{COMPARADOR} \rightarrow > \mid <$

$\text{NOMBRE} \rightarrow \text{IDENTIFICADOR RESTO_NOMBRE}$

$\text{RESTO_NOMBRE} \rightarrow \text{IDENTIFICADOR RESTO_NOMBRE} \mid \lambda$

3.2 Clasificación de la Gramática

La gramática presentada pertenece a la clase de las *Gramáticas Libres de Contexto (GLC)*, ya que cada

producción posee exactamente un único símbolo no terminal en el lado izquierdo de la regla. Además, la estructura definida permite modelar consultas compuestas y expresiones recursivas mediante la utilización del no terminal *RESTO_CONDICION*, encargado de representar secuencias de filtros enlazados por operadores lógicos.

4. Implementación

La implementación se organiza en módulos independientes: *lexer.py*, encargado del análisis léxico; *parser.py*, responsable del análisis sintáctico y la construcción del AST; y *main.py*, utilizado para ejecutar consultas y realizar pruebas del sistema. El lexer reconoce palabras reservadas, identificadores, números y operadores relacionales. Los identificadores se definen mediante la expresión regular $[a-zA-Z_][a-zA-Z0-9_]*$, los números mediante $\backslash d+$ y los operadores mediante los símbolos $>$ y $<$. Los espacios y tabulaciones son ignorados durante el proceso de análisis.

El parser LALR(1) recibe la secuencia de tokens generada por el lexer y verifica que la entrada respete las producciones definidas en la gramática. Cuando una consulta es válida, se genera una representación estructurada de la misma mediante un *Árbol Sintáctico Abstracto (AST)*. En caso contrario, se informa el token inesperado para facilitar la corrección de la entrada.

4.1 Lenguaje de programación

Se eligió Python debido a su simplicidad, legibilidad y amplia disponibilidad de bibliotecas para el procesamiento de lenguajes formales. Para el desarrollo se utilizó la biblioteca *PLY (Python Lex-Yacc)*, la cual permite construir analizadores léxicos y sintácticos mediante expresiones regulares y reglas gramaticales. PLY implementa una estrategia de análisis sintáctico LALR(1), adecuada para el lenguaje definido en este trabajo.

4.2 Analizador Léxico

El analizador léxico tiene la responsabilidad de transformar la cadena de entrada en una secuencia de tokens. Para ello reconoce palabras reservadas como *buscar*, *mostrar*, *ver*, *stock*, *precio* y *categoria*, además de identificadores, valores numéricos y operadores relacionales.

La clasificación de los tokens se realiza mediante expresiones regulares. Cuando una secuencia coincide con el patrón de identificador, se verifica si corresponde a una palabra reservada del lenguaje; en caso contrario, se clasifica como IDENTIFICADOR. Los números son convertidos a valores enteros para facilitar su procesamiento posterior por el analizador sintáctico.

4.3 Analizador Sintáctico

El analizador sintáctico fue implementado mediante la biblioteca PLY utilizando una estrategia LALR(1). Su función consiste en verificar que la secuencia de tokens generada por el analizador léxico respete las reglas definidas por la gramática.

Para ello se definieron producciones que representan las distintas consultas admitidas por el lenguaje, tales como búsquedas de productos, consultas de stock y filtros por categoría o precio. Asimismo, se incorporó una producción recursiva que permite combinar múltiples filtros mediante los operadores lógicos "y" y "o".

A continuación se muestra un fragmento representativo de una producción sintáctica:

```
def p_condicion(p):  
    """ condicion : filtro resto_condicion """  
    p[0] = { 'tipo': 'CONDICION',  
            'filtro': p[1], 'resto': p[2] }
```

Esta regla reconoce una condición compuesta por un filtro principal y una posible continuación de filtros enlazados mediante operadores lógicos.

4.4 Construcción del AST

Durante el análisis sintáctico se construye un Árbol Sintáctico Abstracto (AST), representado mediante estructuras de datos tipo diccionario. Cada nodo almacena información sobre el tipo de construcción reconocida y sus componentes asociados.

Por ejemplo, una consulta de stock genera un nodo con el tipo de operación y el producto solicitado:

```
def p_stock(p):  
    """ stock : VER STOCK IDENTIFICADOR """  
    p[0] = { 'tipo': 'CONSULTA_STOCK',  
            'entidad': p[3] }
```

Para la consulta: *mostrar productos con precio < 15000 y de categoria oversize* el sistema genera una representación estructurada en formato JSON, permitiendo visualizar la organización jerárquica de la consulta y los filtros aplicados. Esta representación facilita futuras extensiones del lenguaje, como la ejecución de consultas sobre una base de datos o un sistema de inventario.

4.5 Manejo de Errores

Se incorporó un bloque dedicado a la captura y reporte de errores sintácticos. Esta característica otorga robustez al sistema, permitiendo identificar la línea exacta y el token que originó la falla estructural.

Fragmento de manejo de errores:

```
def p_error(p):  
    if p:  
        print(f"Error Sintáctico: Token inesperado  
'{p.value}' en la línea {p.lineno}")  
    else:  
        print("Error Sintáctico: Fin de archivo inesperado.  
Faltan tokens por analizar.")
```

4.6 Código fuente del proyecto

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio de GitHub: <https://github.com/prisrodriguezz/tc2026-integrador.git>

5. Casos de prueba y resultados

Entrada	Resultado esperado	Resultado obtenido	Cobertura
buscar producto mate imperial	Válida	Válida	Nombre compuesto
ver stock yerba mate	Válida	Válida	Consulta de stock
mostrar productos sin stock	Válida	Válida	Filtro simple

mostrar productos con stock	Válida	Válida	Filtro simple
mostrar productos con precio > 5000	Válida	Válida	Comparador mayor
mostrar productos con precio < 20000	Válida	Válida	Comparador menor
mostrar productos de categoría mates y con stock	Válida	Válida	Conector y
mostrar productos sin stock o con precio < 1000	Válida	Válida	Conector o
mostrar productos con precio 5000	Inválida	Inválida	Falta comparador
ver producto mate	Inválida	Inválida	Comando no definido
mostrar productos de categoría	Inválida	Inválida	Falta nombre

6. Discusión y análisis

Durante el desarrollo del proyecto se presentaron diferentes desafíos relacionados con el diseño e implementación del lenguaje.

En una primera versión, la gramática utilizaba operadores más formales como “ES”, “MAYOR_A” y “MENOR_A”. Sin embargo, se observó que estas estructuras no resultaban intuitivas para el usuario

final, por lo que se decidió rediseñar el lenguaje utilizando expresiones más naturales y cercanas al lenguaje cotidiano.

Otro desafío importante fue la representación de nombres de productos compuestos por varias palabras. Para resolverlo, se implementó una producción recursiva que permite la concatenación de identificadores, posibilitando la definición de nombres compuestos dentro del lenguaje.

Asimismo, fue necesario incorporar estructuras recursivas en las condiciones, lo que permitió encadenar múltiples filtros mediante los conectores lógicos “y” y “o”.

Durante el diseño de la gramática se identificó además un conflicto en las producciones asociadas al no terminal FILTRO, debido a que dos alternativas compartían el prefijo común “con”. Para resolver esta ambigüedad y mejorar la factorización de la gramática, se introdujo el no terminal RESTO_CON, el cual centraliza las alternativas asociadas a dicho prefijo, permitiendo una definición más clara y estructurada de las producciones.

7. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera integrada los conceptos de gramáticas formales, análisis léxico y análisis sintáctico en la construcción de un lenguaje propio. A través del diseño de la gramática y su posterior implementación, se comprendió la importancia de definir producciones claras y no ambiguas para garantizar un análisis sintáctico correcto. Asimismo, la implementación del analizador permitió evidenciar cómo una especificación formal puede transformarse en una solución funcional, capaz de interpretar y validar consultas de un dominio específico, fortaleciendo la comprensión de los conceptos abordados durante la asignatura.

Bibliografía

[1] D. Beazley, "PLY (Python Lex-Yacc)," [En línea]. Disponible en: <https://www.dabeaz.com/ply/ply.html>.

[2] J. E. Hopcroft, R. Motwani y J. D. Ullman, Teoría de autómatas, lenguajes y computación, 3.^a ed. Madrid, España: Pearson Educación, 2007.